CS 499 Milestone Three: Algorithms and Data Structures

Artifact Enhancement Narrative

Samuel Raymond Kwibe

CS 499 Computer Science Capstone

Southern New Hampshire University

Section One: Describing the Artifact

The artifact I am using for this milestone is the MunchiesSNHU Food Waste Tracker — the same project I worked on in Milestone Two. I originally built this application during an earlier course in the Computer Science program at Southern New Hampshire University. The whole idea behind it was to create something a college campus could actually use to manage food waste more intelligently. The app tracks food waste entries, monitors the condition of compost bins, records environmental sensor data like temperature, humidity, pH levels, and gas readings, and generates reports that staff and administrators can use to make real decisions about waste pickup and sustainability goals.

The project has a frontend built with HTML, CSS, and JavaScript and a backend running on Node.js, Express, and MongoDB. It supports different user roles — staff, admin, and regular users — and has dashboards, data entry forms, and reporting pages. In Milestone Two, I focused on fixing all the structural and integration problems in the backend. I connected routes that were never mounted, moved business logic into proper controllers, cleaned up empty placeholder files, and turned on the security middleware that was installed but sitting dormant. That work got the app functioning correctly. Now in Milestone Three, I took it a step further and made the app actually useful — because collecting data means nothing if the application cannot analyze it and help someone make a decision with it.

Section Two: Why I Chose This Artifact and What I Improved

I chose this artifact for the algorithms and data structures category because I genuinely believe food waste management is an algorithmic problem at its core. Think about it this way — if a campus has fifteen compost bins spread across different buildings, and staff only have time to

service ten of them in a morning, how do they decide which ones to prioritize? Without any logic in the system, someone has to manually check every dashboard reading and compare them in their head. That is slow, error-prone, and exhausting. With a well-designed algorithm, the application can do that comparison automatically and just tell staff exactly where to go first. That is the kind of real-world value that algorithms can deliver, and building it into this project felt meaningful rather than just academic.

Here is a breakdown of everything I improved for this milestone and why each piece mattered:

- The bin pickup priority algorithm in wastePriority.js was the main thing I built for this milestone and honestly the part I am most proud of. The algorithm pulls live sensor readings straight from MongoDB — how full each bin is, the gas concentration levels, the pH reading, the compost weight, and when the last reading was recorded — and uses all of those factors together to calculate a numeric priority score for each bin. I weighted bin fullness the most heavily because a full bin is the most visible and urgent operational problem. Gas levels came next because high gas concentrations mean decomposition is advanced and the bin probably smells bad, which is a health and comfort concern for anyone nearby. Then I factored in pH imbalance because an off-balance pH affects the quality of the compost output. Compost weight and the age of the reading round out the score. Once every bin has a score, the algorithm sorts all of them from highest to lowest, and the dashboard shows staff a clean ranked list of which locations need attention first. Figuring out how to weight those factors was actually harder than writing the code — I had to think carefully about what each factor really meant in practice and what a campus staff member would actually care about most.
- The MongoDB aggregation pipelines in foodWasteRoutes.js were another big piece of this milestone. The reporting endpoint at /api/waste/reports/summary uses MongoDB's $group and $sum operators to calculate total waste per day, total waste per location, average bin fullness across the whole campus, and compost diversion percentage — all pulling from real database records, no fake numbers anywhere. I learned a lot building

these pipelines because they require thinking about data in a completely different way than regular queries do. Instead of asking the database for individual records, I am asking it to transform and summarize a whole collection into something meaningful. That kind of thinking — grouping data, calculating aggregates, shaping the output for a specific purpose — is directly connected to what data structures and algorithms are really about.

- I also added defensive data handling throughout app.js, and I want to be honest — this was something I did not think much about until I actually ran the app with an empty database and watched it crash. The original code assumed the API would always return data. If the sensor array came back empty, the code would try to access the last element of an empty array, get undefined, and then crash trying to read a property off of undefined. I had to go through every single place in the frontend where data was being accessed and add checks to make sure the array existed, had at least one item in it, and that the expected fields were actually present before trying to use them. It sounds tedious but it taught me something important about writing algorithms that work in the real world rather than just the ideal case.
- The chart optimization was another improvement I made in app.js. When I first looked at how the charts were being updated, I noticed they were being completely destroyed and rebuilt from scratch every ten seconds whenever new data came in. Every refresh meant deleting the canvas element, creating a new one, reinitializing the entire Chart.js object, and re-rendering every single data point. I asked myself why that was necessary if the structure of the chart never changes — only the data values do. So I refactored the rendering logic to keep the chart instances alive and just update their data arrays using Chart.js's built-in update method. The result is faster, cleaner, and stops the screen from flickering every time new data loads. It was a small change in terms of lines of code but a meaningful improvement in terms of efficiency.
- Finally, I fixed the broken API URL in app.js. The original code was pointing to http://localhost:8000/data but the backend runs on port 5001 and that route does not even

exist. This meant every chart and dashboard feature was failing silently — no data was ever being loaded, no errors were being shown, it just quietly did nothing. Fixing the URL to point to /api/waste was what made it possible to actually test everything else in this milestone with real data from the database.

Section Three: Course Outcomes and Whether I Met Them

Going back to the plan I laid out in Milestone One during the code review, my main goal for this milestone was to demonstrate Outcome Three — designing and evaluating computing solutions that solve a given problem using algorithmic principles and computer science practices and standards, while managing the trade-offs involved in design choices. I am confident I met this outcome through the priority algorithm, the aggregation pipelines, the defensive handling, and the chart optimization work.

The trade-off decision I am most proud of is the one I made when designing the priority algorithm. I actually considered using a simple machine learning model to predict which bins needed service based on historical patterns. It sounded cool and more impressive on paper. But I decided against it for a few reasons. First, the application does not have enough historical data yet to train a meaningful model. Second, a machine learning model would be a black box — staff would see a priority score but have no idea how it was calculated, which makes it hard to trust. The weighted scoring approach I chose instead is completely transparent. Anyone can look at the weights and understand exactly why a particular bin got a high score. In a real operational environment where people have to act on the system's recommendations, that kind of explainability matters a lot. That decision — choosing a simpler but more trustworthy algorithm over a more complex but opaque one — is exactly the kind of design trade-off that Outcome Three is asking students to demonstrate.

I also made progress on Outcome Four through the MongoDB aggregation pipelines, which represent a professional use of database query design for analytical purposes. And Outcome Five around security mindset was reinforced through the defensive data handling — validating input before processing it is both a security practice and an algorithmic one.

My outcome coverage plan does not need any changes. The database-specific work I planned for Milestone Four — schema validation, numeric range enforcement, and database indexing — will take care of the remaining portions of Outcome Four.

Section Four: What I Learned and What Was Hard

I want to be honest about how this milestone felt going in. After the structural work of Milestone Two, I was actually excited about Milestone Three. Fixing routes and cleaning up controllers is important but it is not particularly creative work. Algorithms felt like the part where real problem-solving happens — where the question is not just whether the code runs correctly but whether it actually does something intelligent and useful.

Designing the bin priority algorithm was genuinely fun but harder than I expected. Writing the code was not the hard part. The hard part was deciding how to weight the factors. I sat with that question for longer than I expected to. Bin fullness felt like the obvious top priority — a full bin is an immediate visible problem. But gas levels also matter a lot because they affect people nearby. And the age of the reading matters because stale data means nobody has checked that location recently. I ended up sketching out a few different weighting scenarios on paper and thinking through what each one would look like in practice before settling on the final approach. That process of working through design decisions before writing a single line of code is something I have heard about in software engineering courses but this is the first time I really felt the value of it firsthand.

One thing that genuinely surprised me was how much easier Milestone Three became because of the work I did in Milestone Two. When I started building the priority algorithm, the routes were already clean and protected, the data models were correctly structured, and the database connection was solid. I did not have to stop and fix infrastructure problems while trying to think through algorithmic logic. That experience made the relationship between Milestone Two and Milestone Three feel very real — good software design from the first milestone directly created the conditions for better algorithmic work in the second one. I did not fully appreciate that connection until I was living it.

The defensive data handling taught me a lesson I will not forget. I genuinely thought adding empty array checks was going to be quick and simple. Then I actually ran the app against an empty database and watched multiple things break in ways I did not expect. I had to trace through the code carefully and think about every state the data could possibly be in — not just the normal case where everything works but all the edge cases where it does not. That kind of systematic thinking about failure modes is something that I now see as a fundamental part of writing good algorithms, not just an extra step to do at the end.

The chart optimization was satisfying in a different way. It was one of those moments where I looked at something that was technically working and asked whether it was working well. Rebuilding a chart from scratch every ten seconds works — the user gets updated data. But it wastes a significant amount of browser resources doing work that does not need to be done. Making that change felt like leveling up from writing code that works to writing code that works efficiently, and that distinction feels important for where I want to go in my career.

Overall, Milestone Three is the point in this project where I feel like the MunchiesSNHU application became something real. The priority ranking on the staff dashboard gives a direct, actionable answer to the question that campus waste management staff face every single day — which bins need attention right now and in what order. That is a small thing in the context of a school project. But it represents exactly the kind of value that thoughtful algorithm design can deliver in a real working environment, and building it is something I am genuinely proud of.

AI Usage Disclosure

An AI writing assistant was used to help organize and draft portions of this narrative. All content was reviewed, edited, and rewritten to reflect my own understanding, direct experience working through the artifact enhancements, and honest reflection on the challenges and lessons I learned throughout this milestone.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

MongoDB. (2023). Aggregation pipeline. MongoDB documentation. https://www.mongodb.com/docs/manual/core/aggregation-pipeline/

Chart.js. (2023). Updating charts. Chart.js documentation. https://www.chartjs.org/docs/latest/developers/updates.html